Modbus RS485 --- RTU --- Serial port communication

History

Modbus = Modicon (now Schneider Electric's) + Fieldbus
Modbus = the most popular protocol + the most used fieldbus standard.

Once upon a time, all sensors, transmitters, motors, etc had to be connected directly to the DCS. So, when you have a big plant, a lot of cabling had to be done to be able to connect all the machines to the DCS (Distributed control system = Main control panel)
Modbus is a solution to the many (high cost) cabling and connecting. Modbus enables you to communicate between different local control panels (for example PLC).

1. RS232 (mode of operation: half or full duplex)

Before the USB was used on computers, they used RS232. This type could also b
communication between different local control panels (=computers). But there a

- Point-to-point (only one device to another device → no network)
- Maximum distance is limited to 15m (50ft)
- Not so resistance against electrical noise

2. RS485 (mode of operation: half duplex only, because multiple slaves possible)

Advantages compared to the older RS232 communication method:

- Point to multipoint (multiple devices, up to 32, could be connected → network)
- Distance up to 1300m (4000ft)
- It is build that way that it is very resistance to electrical noise (twisted wiring)

3. Important difference Modbus/RS485

Modbus: Protocol which defines a messaging structure to exchange data
RS485: Electrical standard which defines the electrical signal levels and wiring that allows data to be transferred (≠ protocol). It is the media on where the messages are transferred / the way on which the protocol goes.

4. Modes of operation

Commonly known modes of operation (both using master/slave configuration):
-**ASCII** (American Standard Code for Information Interchange): disappeared
-**RTU** (Remote Terminal Unit): much faster and efficient as ASCII

This lecture deals about: **Modbus protocol implemented as RTU mode over RS485**

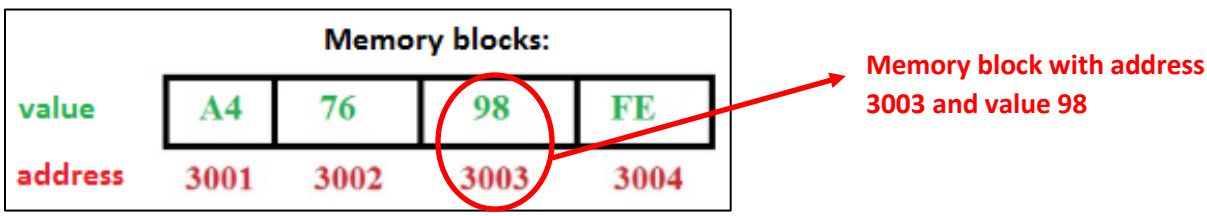
To be enable to communicate over Modbus protocol, each device (for example PLC) need the below **3 components**:

1. CPU
2. Memory
3. Communication interface (for example RS485)

Memory

Memory is stored in blocks; each block has its unique address and its own value:
Value: the actual value stored in the memory block. (for example, suction pressure input)
Address: a number attached to the memory block to allow that block to uniquely identified and also find rapidly.

→ *when the value is changing (for example pressure change), the address stays unchanged (=unique)*



Memory block Sizes

Depends on the purpose of the block

Bit = Binary + Digit (0 or 1)		
1	ON/OFF	
8	11101000 = 232	Max 255 (or -127/+127) – For example: analog valve
16	…	
32	…	

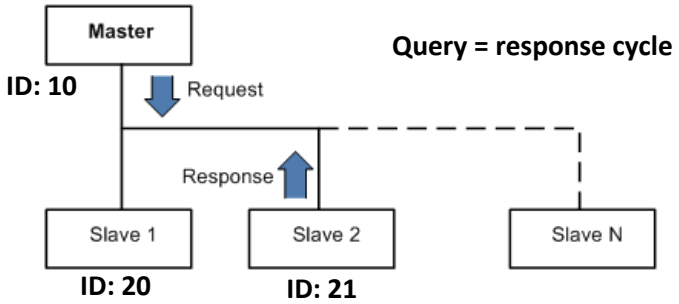
Modbus Memory Area

Memory that is reserved/dedicated to Modbus protocol, this area is divided into 4 sub-area's:

Memory block address	**Sub-area**		
1-10000	Coils	Discrete (digital) output	1-bit memory block
10001-20000	Inputs	Discrete (digital) input	
30001-40000	Input Registers	Analog input	16-bit memory block
40001-50000	Holding Registers	Analog output	

Memory block address needed for communication between PLC's

To be able to communicate between PLC's, there is always a MASTER plc and (multiple) slaves. The master plc will ask for information to the slaves. Since the communication is set into the open, meaning the request will get to all the slaves, each slave will need to configured with their own ID.



There **6 different <u>requests</u>** (commands) the master plc can give:

Function code	Command	
1	Read coil status	READ
2	Read input status	
3	Read holding registers	
4	Read input registers	
5	Force single coil	WRITE
6	Preset single register	
7	Read Exception status	
15	Force multiple coils	
16	Preset multiple registers	
17	Report Slave ID	

Request example →

Read input status (function code 2)
Modbus Unit ID = 21
Start Memory block address: 10006
Number of blocks = 2

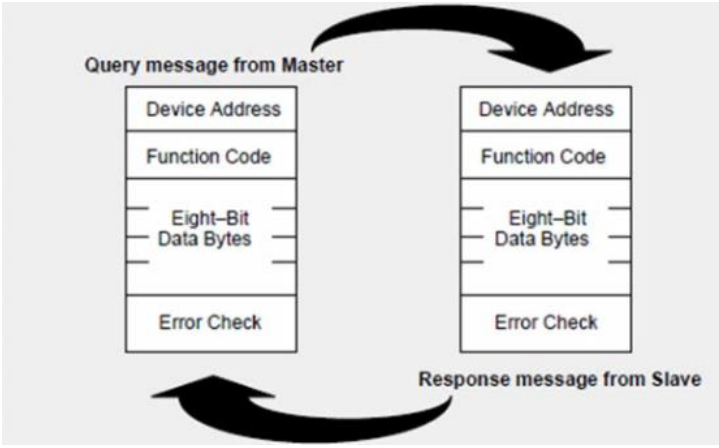
In above request example, the master plc (ID: 10) will ask for an input status of block address 10006 AND 10007 (=number of blocks) of slave number 2 (ID: 21)

To be able to communicate over a serial port in RS485 network, the below settings should be the same for all the devices in the network:

Setting	**Description**	**Mostly used**
Baud rate (bps)	How fast the bits are transmitted on the line	9600 bps
Number of data bits	Possibilities: 7-8	8
Number of stop bits	Possibilities: 1-2	1
Parity	Error checking. Possibilities: none/odd/even	none

How does the query-response cycle works?

The master will send a request (query) in the form of a **byte stream**, the slave will respond by similar (but different) **byte stream**, with 4 different sections consisting of bites, who serves different purposes:



4 different sections consist of bites, will be send via a byte stream from/to master/slave plc

From the view of the Master (query) → see above

Device Address	Unit ID of Targeted Slave Device (1 Byte)	**Modbus unit ID**
Function Code	Tells The Slave What Function To Perform (1 Byte)	**Function code**
Eight-Bit Data Byte	Specifies Address Block Range (4 Bytes)	**Start Memory block address + Number of blocks**
Error Check	CRC (Cyclical Redundancy Check) for Data Transmission Error Detection (2 Bytes)	**[X][X]** **Represented as "XX" because you cannot manually calculate this value (and it is not needed)**

Eight-bit Data Byte:

---**Start memory block address – lower range of memory block address**.

(for example 30019, range = 30001-40000 → 30019 – 30001 = 18)

---**Number of blocks**: 1

[0][18][0][1]

2 + 2 = 4 bytes.

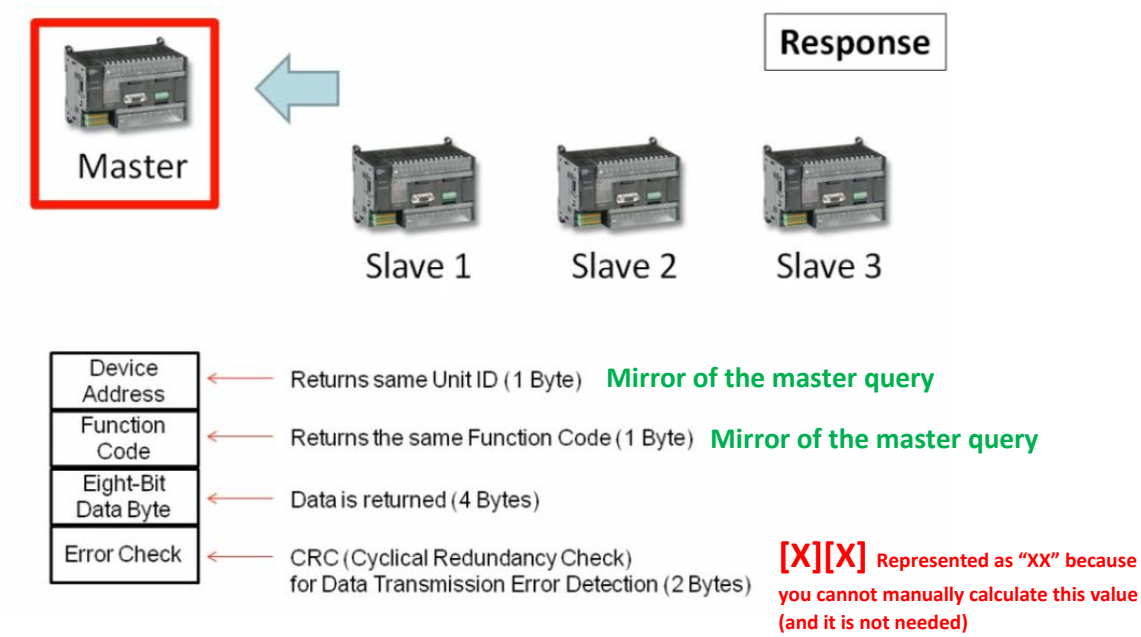
Each byte represent max 256 decimals, that's why "0" in first byte

From the view of the slave (response):

As example we take the query as below by the master:
---The master plc wants to know (read input register, function code **4**) only the local pressure located in address block **30019** of slave plc **22**

[22][4][0][18][0][1][X][X]

What is the response of the slave?



Eight-bit Data Byte:

The data the slave will be sending back to the master plc: **35 psi**

[2][0][35] (single 16 bit value)
2: meaning 2 bytes will follow
0: high byte
35: low byte

Response of the slave: **[22][4][2][0][35][X][X]**

Read input status

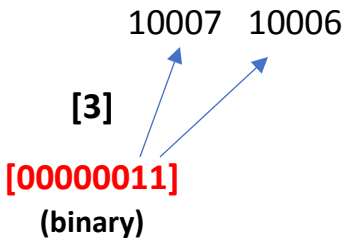
Example we want the position of 2 local valves located at a slave plc 21. The position of both valves are located in address blocks 10006 and 10007 (both closed = value 1). We will use for this the read input status command (function code 2). The input will be open/close.

Query byte stream: **[21][2][0][5][0][2][X][X]**

21: slave plc ID 21
2: function code "read input status"
0 5: start address block 10006 – 10001 = 5
0 2: Number of address blocks we want
X X: CRC (error check)

Response byte stream: **[21][2][1][3][X][X]**

21: mirror master
2: mirror master
1: indicates how many bytes will follow
3: position of the valve
X X: CRC (error check)

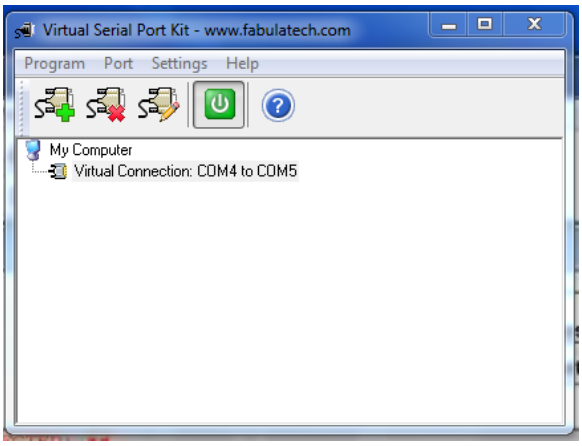


SOFTWARE

With the below software you can simulate RS485 communication commands and byte streams. It is useful for troubleshooting.

Virtual Serial port kit

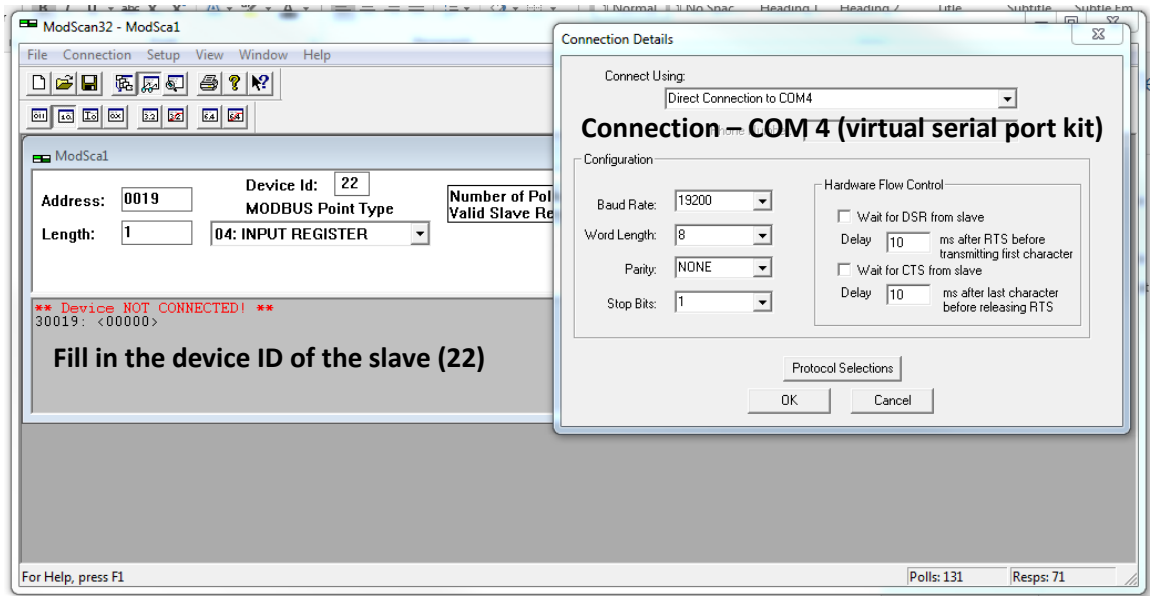
Setup a virtual Modbus network where Modscan 32 (Master) exchanges data with Modsim 32 (Slave).
→ Make virtual (Fake) COM-ports



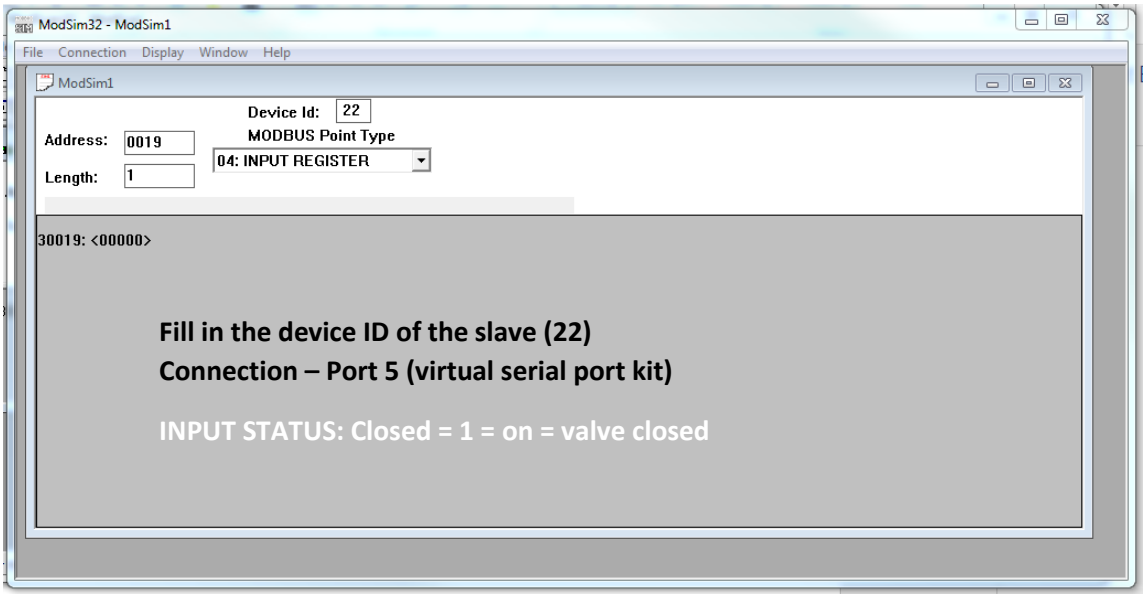
COM 4: Master

COM 5: Slave (connect to port 5)

Modscan 32 – MASTER



Modsim 32 - SLAVE



CAS Modbus Scanner (www.chipkin.com) -- uses hexadecimal (HEX) presentation!!

Since software Modscan 32 (Master) is not able to generate WRITE functions (only read functions), another software is introduced to simulate the write functions. This software for master can be used in combination with Modsim 32 (Slave)

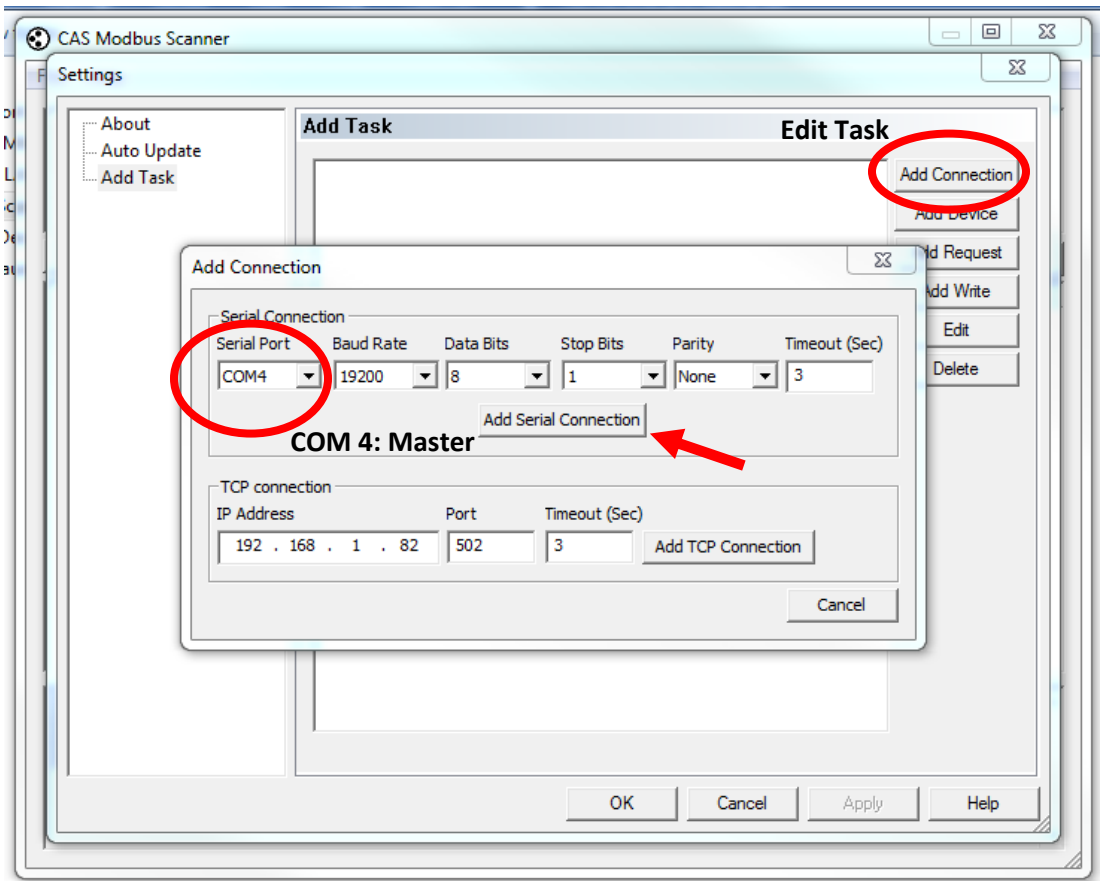
Master - Query - Force single coil – function 5 – digital output

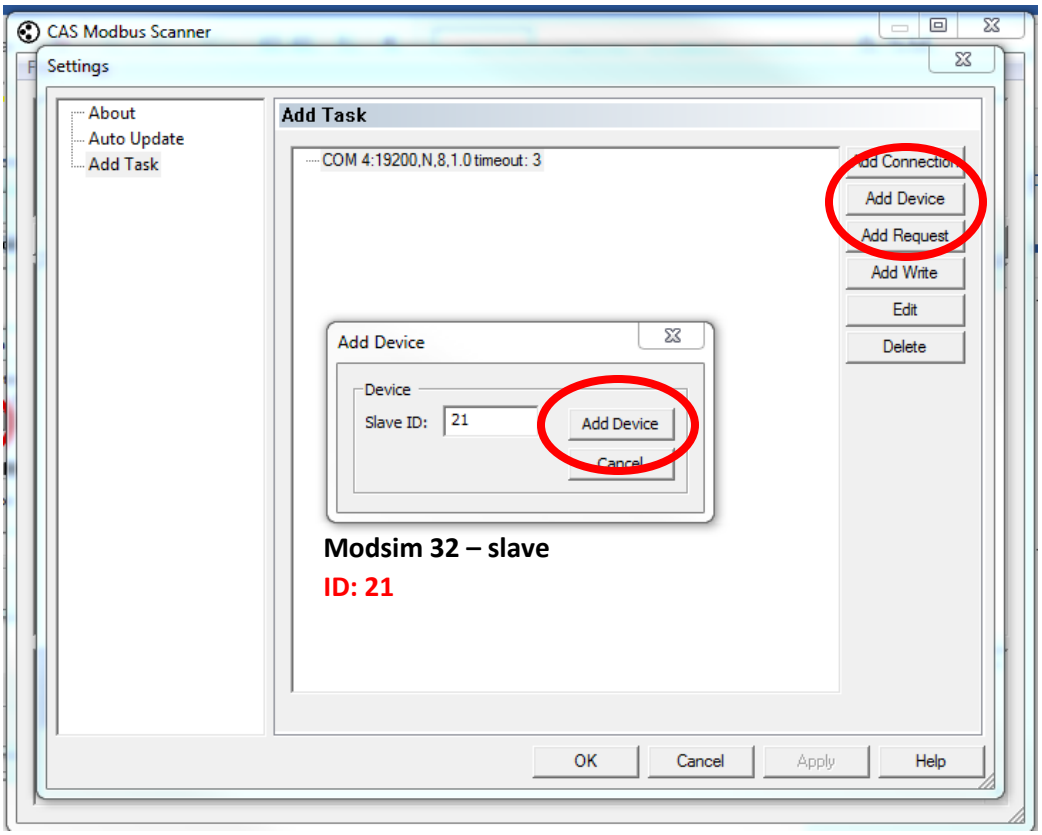
1. To set a coil to "1" → [21][5][0][9]**[255][0]**[X][X] ---- **HEX: [15][05][00][09][FF][00][X][X]**
   Slave response: [21][5][0][9][255][0][X][X] ----- **HEX: [15][05][00][09][FF][00][X][X]**
   = IDENTICAL

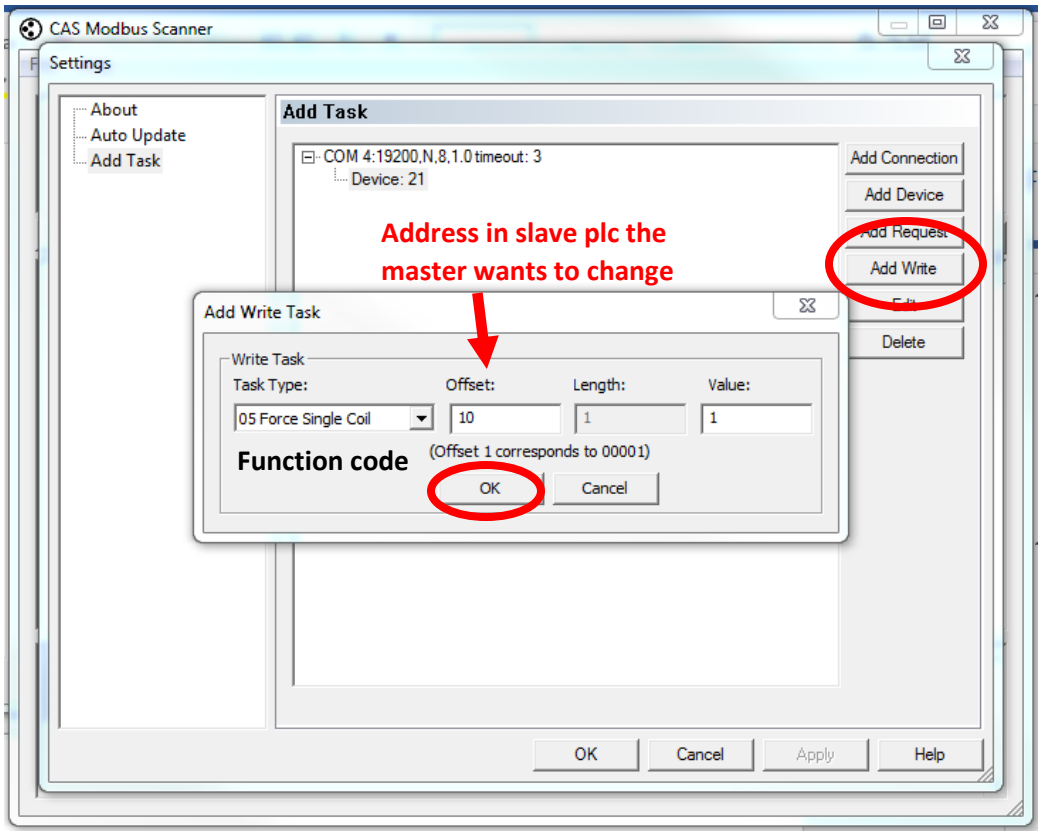
= Modbus convention

2. To set a coil to "0" → [21][5][0][9]**[0][0]**[X][X]
   Slave response: [21][5][0][9][0][0][X][X]
   =IDENTICAL

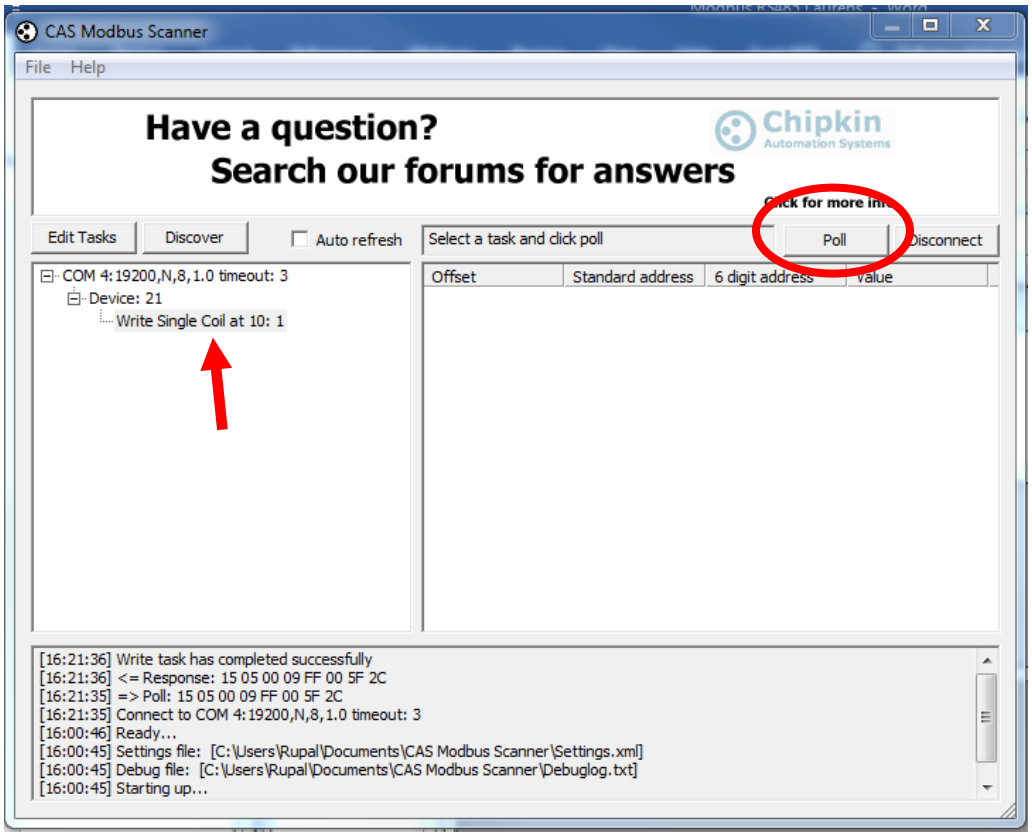
So, if the write command is successful, the slave plc sends a response which is identical (bite per bite) to the master query.




CAS Modbus Scanner
Settings
About
Auto Update
Add Task
Add Task
COM 4:19200,N,8,1.0 timeout: 3
Add Connection
Add Device
Add Request
Add Write
Edit
Delete
Add Device
Device
Slave ID:
21
Add Device
Cancel
Modsim 32 – slave
ID: 21
OK
Cancel
Apply
Help


CAS Modbus Scanner
Settings
About
Auto Update
Add Task
Add Task
COM 4:19200,N,8,1.0 timeout: 3
Device: 21
Add Connection
Add Device
Add Request
Add Write
Delete
Address in slave plc the
master wants to change
Add Write Task
Write Task
Task Type:
Offset:
Length:
Value:
05 Force Single Coil
10
1
1
Function code
(Offset 1 corresponds to 00001)
OK
Cancel
OK
Cancel
Apply
Help

To write the digital output, select the command and click “Poll”. The digital output in Modsim32 (slave) will change from “0” to “1”.



<u>Master - Query – Preset single register – function 6 – analog output</u>

This function is for example to open/close a valve (0-100%).

If we want to open the valve to 50:

Query → [22][6][0][20][0][50][X][X] ---- **HEX: [16][06][00][14][00][32][X][X]**

Slave response: [22][6][0][20][0][50][X][X] ----- **HEX: [16][06][00][14][00][32][X][X]**

= IDENTICAL

Modbus Exception codes (responses) --- see MODBUS Exception Codes

These codes are useful to trouble shoot communication errors.

Imagine the Master plc wants to read an analog input which doesn't exist, the master query is for example: [22][4][0][23][0][1][X][X]

- 22: ID address of slave plc
- 4: function code Read input registers
- 0 23: Memory block address 30024 (which doesn't exist in this example)
- 0 1: Number of address blocks we want

Since this memory block address doesn't exist in the slave plc, the response will be: [22][132][2][X][X]

- 22: ID address of slave plc
- 132: Error designation → ***128*** + function code (4)
- 2: Error code: ILLEGAL DATA ADDRESS

Floating points

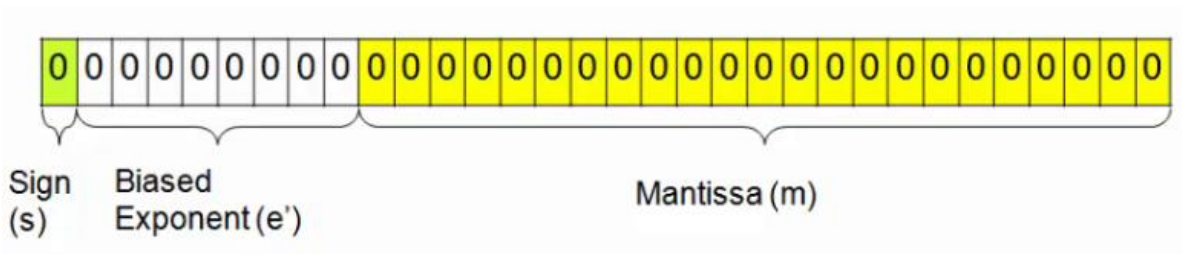
- 16-bits can be 0000000000000000 or 1111111111111111 → decimal 0 to 65535
  This means that Modbus can't support values above 65535
- The Modbus only supports integer values (whole numbers), so no numbers with comma.

Solution = Floating Point

1. Double Precision floating point
2. Single Precision floating point
   Most frequently used following standard IEEE754

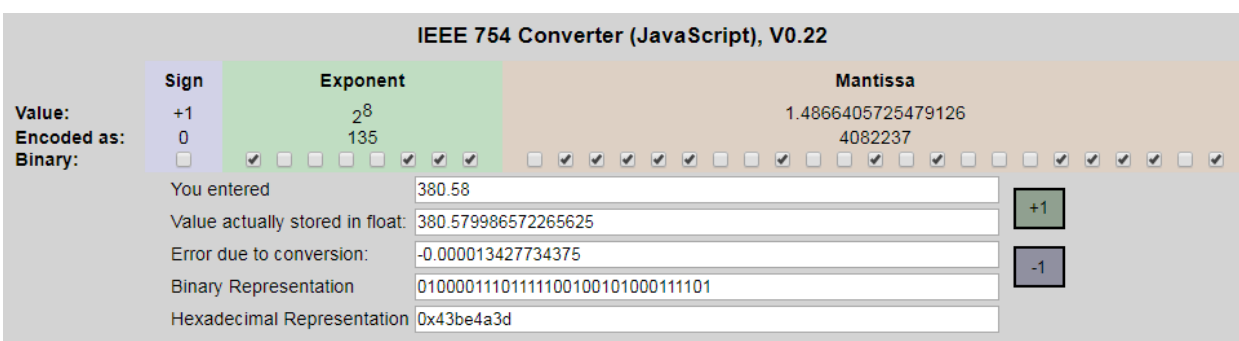
   Standard Modbus 16-bit = single value
   Single Precision Floating Point: 16-bit + 16-bit = single value
   → So, the floating point is represented by a **32-bit**

   To represent the floating-point number, this 32-bit is sorted in a different way and the bit groups have different purposes.



In the below website, you can convert decimal number into hexadecimal number:
https://www.h-schmidt.net/FloatConverter/IEEE754.html

Imagine the Master plc wants to read an analog input, **floating point** from a slave plc with a value of **380.58**. The Modbus uses this hexadecimal equivalent for data communication of floating points.



HEX: 0x 43 b4 4a 3d

The master query is for example: [01][04][00][0b][00][02][X][X]

- 01: ID address of the slave plc
- 04: function code “read input register’
- 00 0b: Memory block address 30012 (**HEX 0b** = decimal 11)
- 00 02: number of address blocks we want. Combined data to express one value (= floating point). So, one value will be stored under 2 memory block addresses 30012 & 30013

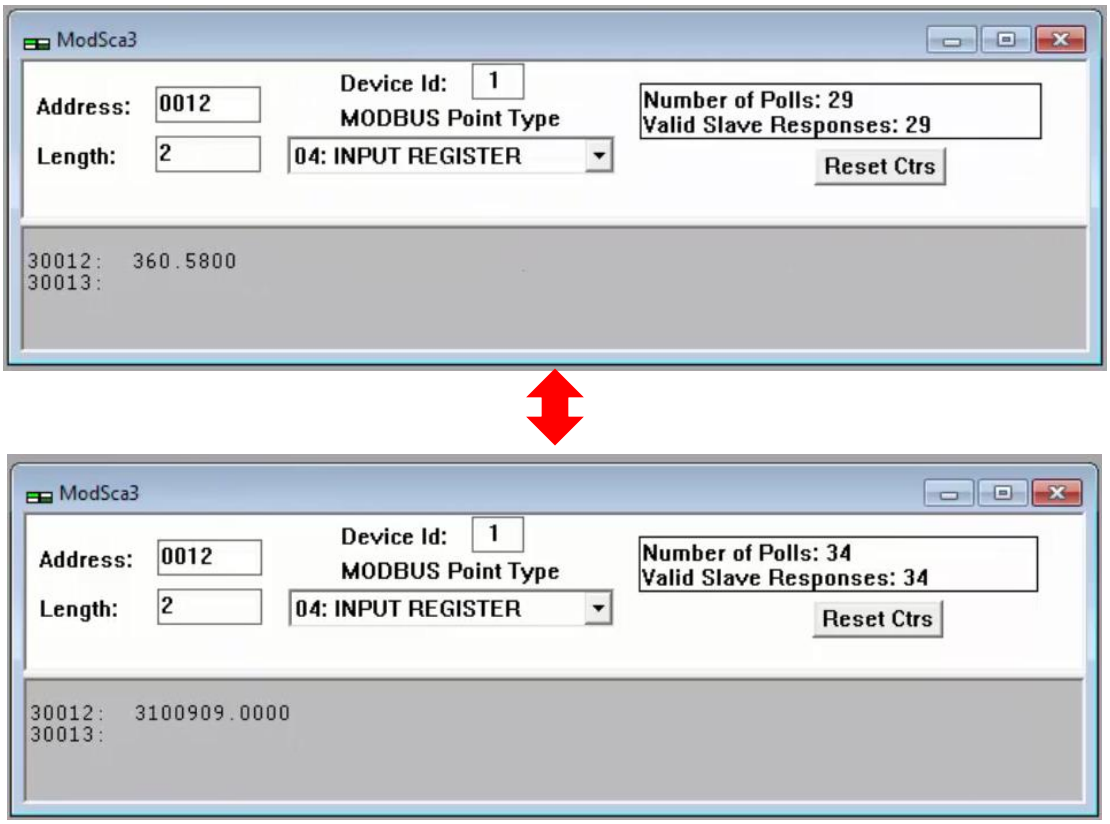
The response of the slave plc: [01][04][04][4a][3d][43][b4][X][X]

- 01: ID address of the slave plc
- 04: function code “read input register”
- 04: indicates how many bytes will follow
- 4a 3d 43 b4: data input value 380.58 in HEX (little Indian format)

Little Indian format: lower 16-bit value (4a 3d) will be send before the higher 16-bit value (43 b4)
= Floating point

Big Indian format: higher 16-bit value (43 b4) will be send before the lower 16-bit value (4a 3d)
= Swapped floating point

Be aware that all the devices (plc) in the field are configured in the same language as it comes to floating points. (Big Indian format or Little Indian format). Otherwise the data transfer will be wrong. (floating point or swapped floating point).



Configure a device for Modbus

Common steps:

1. Locate the appropriate communications port (RS232, RS485, ethernet, etc.) and **configure** it for the Modbus protocol.

2. Get the Modbus memory map for the device.
   What discrete (digital) inputs are mapped to what bit registers (and memory blocks) in the plc so you know which one to read to get the data out of the device you want to communicate with (slave plc).

   Example of memory map:

Discrete Data Types				
DL05 Memory Type	**PLC Range (Octal)**	**Address (484 Mode)**	**Address (584/984 Mode)**	**Modbus Data Type**
Global Inputs (GX)	GX0-GX1746	1001 - 1999	10001 - 10999	Input
	GX1747-GX3777	---	11000 - 12048	Input
Inputs (X)	X0 – X1777	---	12049 - 13072	Input
Special Relays (SP)	SP0 – SP777	---	13073 - 13584	Input
Global Outputs (GY)	GY0 - GY3777	1 - 2048	1 - 2048	Output
Outputs (Y)	Y0 – Y1777	2049 - 3072	2049 - 3072	Output
Control Relays (CR)	C0 – C3777	3073 - 5120	3073 - 5120	Output
Timer Contacts (T)	T0 – T377	6145 - 6400	6145 - 6400	Output
Counter Contacts (CT)	CT0 – CT377	6401 - 6656	6401 - 6656	Output
Stage Status Bits (S)	S0 – S1777	5121 - 6144	5121 - 6144	Output

In the above table the discrete (digital) inputs X are dedicated in memory block address **12049 – 13072.** The discrete (digital) outputs Y are listed in memory block address **2049-3072**